

Chapter 6 — Data Visualization

Chapter 6 — Data Visualization

Charts in React are about data shape, not chart libraries. This chapter teaches you Recharts — but more importantly, it teaches you the *transform-then-render* pattern that applies to every charting library you'll ever touch.

What you'll learn

- The mental model that makes Recharts feel easy: **the chart is a function of the data shape**.
- How to **reshape teams into chart-ready arrays** with `.map()`, `.filter()`, `.reduce()`.
- **Stacked vs grouped bars** — how a single prop (`stackId`) flips between them.
- `<Cell>` — per-bar color overrides for highlighting one row.
- **Pivot transforms** — going from “team 🏠 picks” to “match 🏠 backers” without losing your mind.
- When **internal state** (component-local `useState`) is correct vs lifting it.
- The **chart card pattern** — wrap every chart in a consistent container.

Lessons in this chapter

1. [01-recharts-fundamentals.md](#) — What Recharts gives you, the components you actually use, the responsive container pattern, and the chart card wrapper.
2. [02-reshaping-data.md](#) — Three concrete transforms: stacked-bar totals, accuracy %, finalist pick distribution. Plus per-bar `<Cell>` colors.
3. [03-pivots-and-internal-state.md](#) — The `<LeagueAgreementChart>` case study. Pivoting from team-data to match-data. Why the chart's tab state is local, not global.

Files you'll create or update

```
components/  
├── analytics/  
│   ├── ChartCard.tsx           ← container wrapper for every chart  
│   ├── InsightCard.tsx        ← summary tiles below the charts  
│   └── LeagueAgreementChart.tsx ← the pivot chart with internal state  
└── tabs/  
    └── AnalyticsTab.tsx        ← UPDATED: real implementation
```

New React/Next.js concepts introduced

- **Render-prop / children-as-elements** with Recharts.
- **<ResponsiveContainer>** – the only way to size a chart in a flexible layout.
- **Component-local state** for UI-only toggles inside a chart.
- **Tree-shakable Recharts imports** (only the bits you use).
- **Conditional <Bar> rendering** based on data state.

How long it should take

- 30–45 min reading per lesson
- 1.5–2 hours typing the code
- ~4 hours total

Why charting deserves its own chapter

Three reasons:

1. **Every senior interview eventually asks “build me a dashboard.”** Knowing how to wire a charting library to live data without copy-paste-driven panic is a core senior skill.
2. **Most beginners learn one chart library by rote and then crumble when they switch.** The mental model in this chapter (“data shape, not library”) transfers directly to D3, Victory, Visx, Plotly, anything.
3. **Charts force you to confront *transforms*** – `.map()`, `.reduce()`, `.filter()` – at scale. The data-shaping muscles built here are useful far beyond charting.

Before you start

You should have:

- Chapter 5 complete: Players tab and PlayerDetail working.
- All tabs except Analytics rendering real content.
- recharts already in package.json (Chapter 2 set it up).

Open [01-recharts-fundamentals.md](#).